深度學習
Deep Learning

Recurrent Neural Networks (RNN)

Instructor: 林英嘉 (Ying-Jia Lin)
2026/10/28

Course GitHub

Slido # DL1028

Outline

- Introduction to Natural Language Processing (NLP) [30 min]
- Recurrent Neural Networks (RNN) [50 min]
- Midterm Exam Reminder [5 min]
- NLP code [50 min]
- Quiz [15 min]



什麼是自然語言處理？

文字搜尋



下一個字預測



The Revolution of ChatGPT

ChatGPT came out in November, 2022.

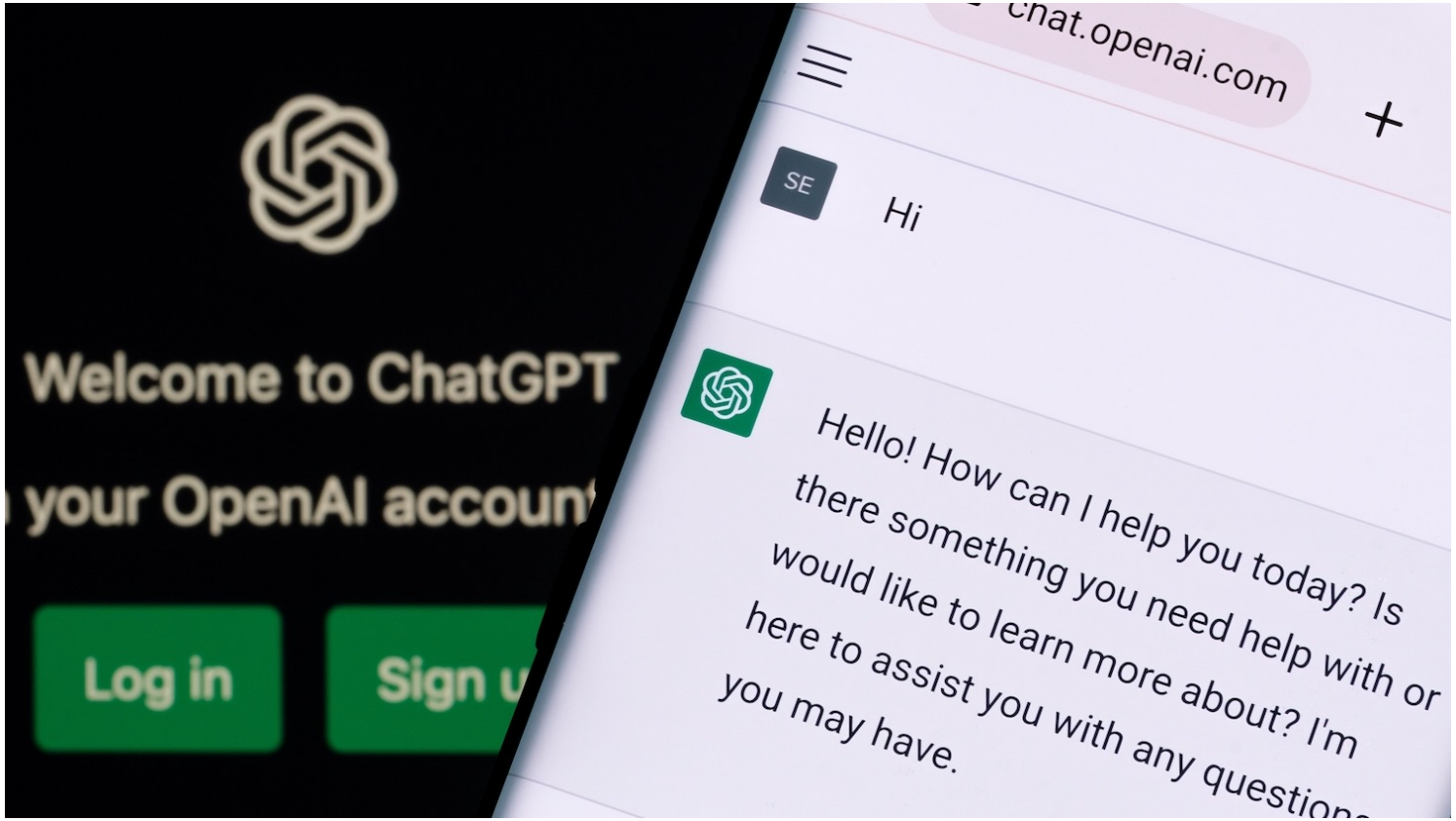


Figure source: https://poole.ncsu.edu/thought-leadership/article/lets-chat-about-chatgpt/

大型語言模型的應用





翻譯、聊天機器人、寫故事、
摘要、教學、
寫程式、寫作業 ...



Code Llama

https://arxiv.org/abs/2308.12950

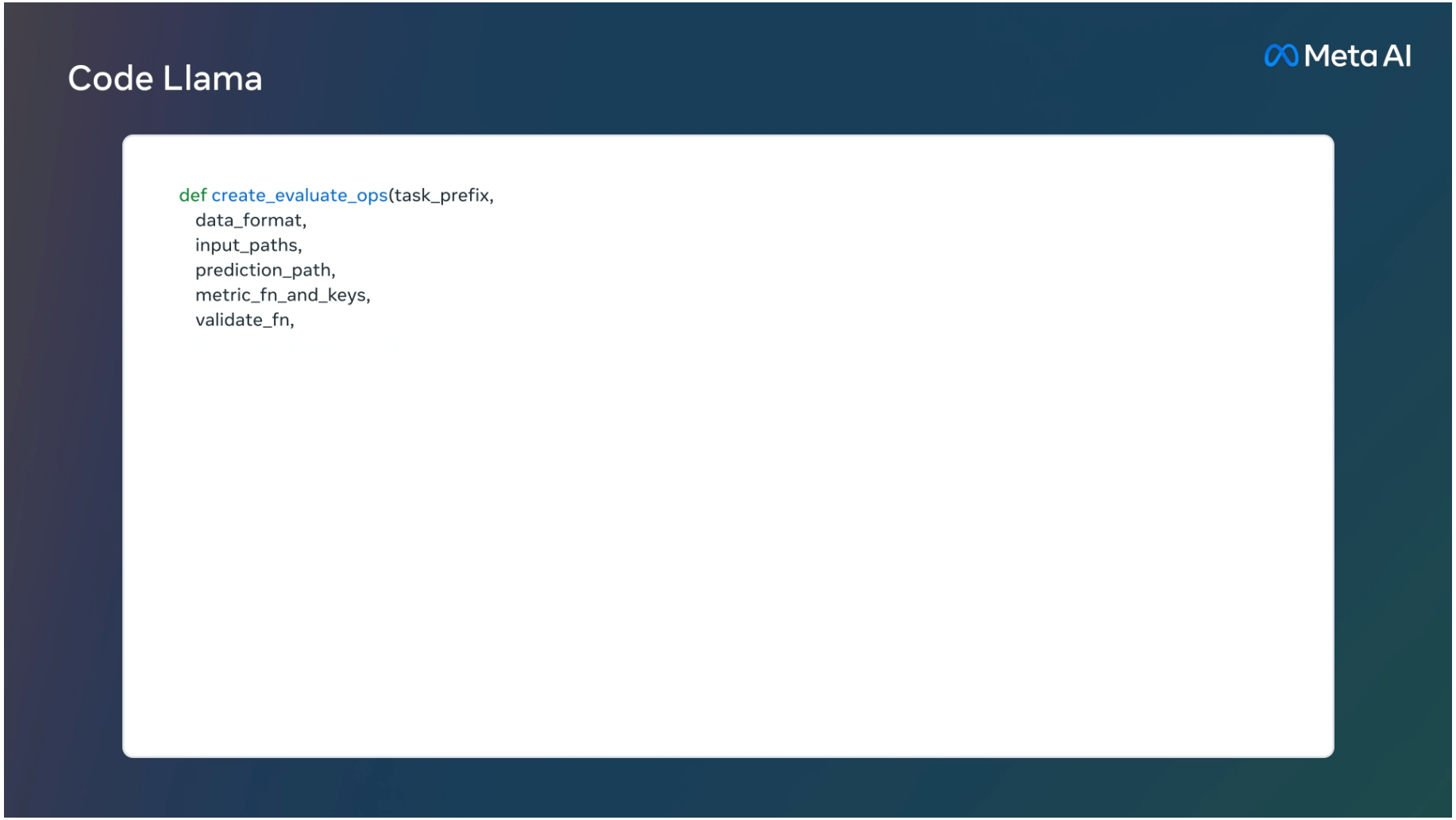
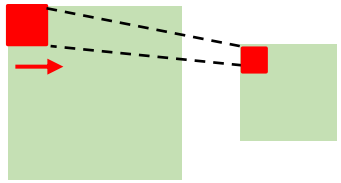


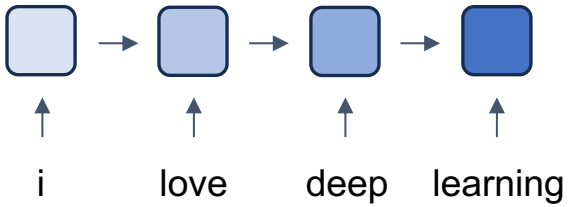
Figure source: https://ai.meta.com/blog/code-llama-large-language-model-coding/

深度學習常見模型架構

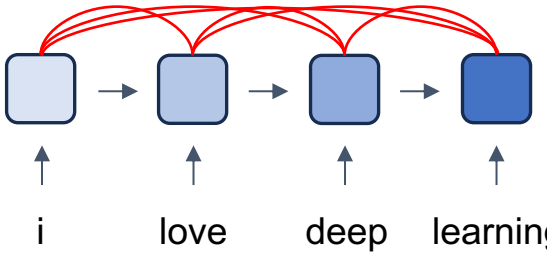
CNN
(著重局部資訊)



RNN
(著重記憶力)



Transformer
(著重全局資訊與記憶力)



Natural Language Processing (NLP)

電腦是如何學習自然語言呢？

NLP 三大秘辛

1. 每個字，都是一段向量
2. 每個 NLP 模型都有一個字典 
3. 自然語言處理模型需要記憶力 🧠

影像 vs. 文字



⟶ 28 x 28 的矩陣

Apple ⟶ ?

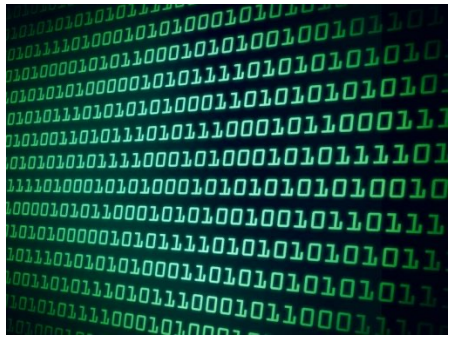


Figure source: https://kopu.chat/wp-content/uploads/2017/04/binary-507x380.jpeg

每個字，都是一段向量

可以自行設定向量長度

she = [0.110960 0.032115 -0.004809]

her = [0.110950 0.032117 -0.004909]

grandpa = [0.023115 0.708091, 0.512345]

grandma = [0.023113 0.708101, 0.512445]



Word Representations

like

apple



?

Basic Approach

Corpus

I like to eat apple

You like to watch TV

Vocabulary

apple	[1, 0, 0, 0, 0, 0, 0, 0]
eat	[0, 1, 0, 0, 0, 0, 0, 0]
I	[0, 0, 1, 0, 0, 0, 0, 0]
like	[0, 0, 0, 1, 0, 0, 0, 0]
watch	[0, 0, 0, 0, 1, 0, 0, 0]
to	[0, 0, 0, 0, 0, 1, 0, 0]
⋮	

One hot encodings with each vector size equal to the vocab size (8 in this case)

Pros: handy, training-free
Cons: sparse matrix which occupies much memory



nn.Embeddings

```
1 word_to_idx = {"好吃": 0, "棒": 1, "给力": 2}
2 embeds = torch.nn.Embedding(3, 5)  # 2 words in vocab, 5 dimensional embeddings
```

```
1  lookup_tensor = torch.tensor(
2      [
3          word_to_idx["好吃"],
4          word_to_idx["棒"],
5          word_to_idx["给力"],
6      ],
7      dtype=torch.long,
8  )
9  word_embed = embeds(lookup_tensor)
10 print(word_embed)
```

[7]

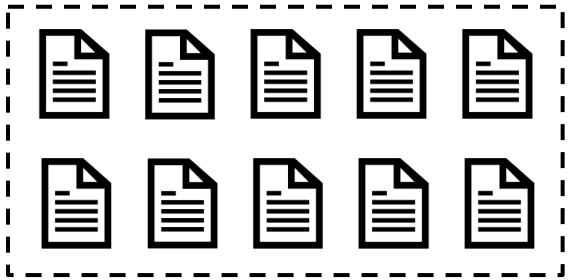
```
tensor([[ 2.2014,  1.9880,  1.5726, -0.1552,  0.0475],
        [-0.0540, -1.5926, -0.5808, -0.6631, -0.3321],
        [-0.3562,  1.0019,  0.8178, -0.9174, -0.0707]],
       grad_fn=<EmbeddingBackward0>)
```

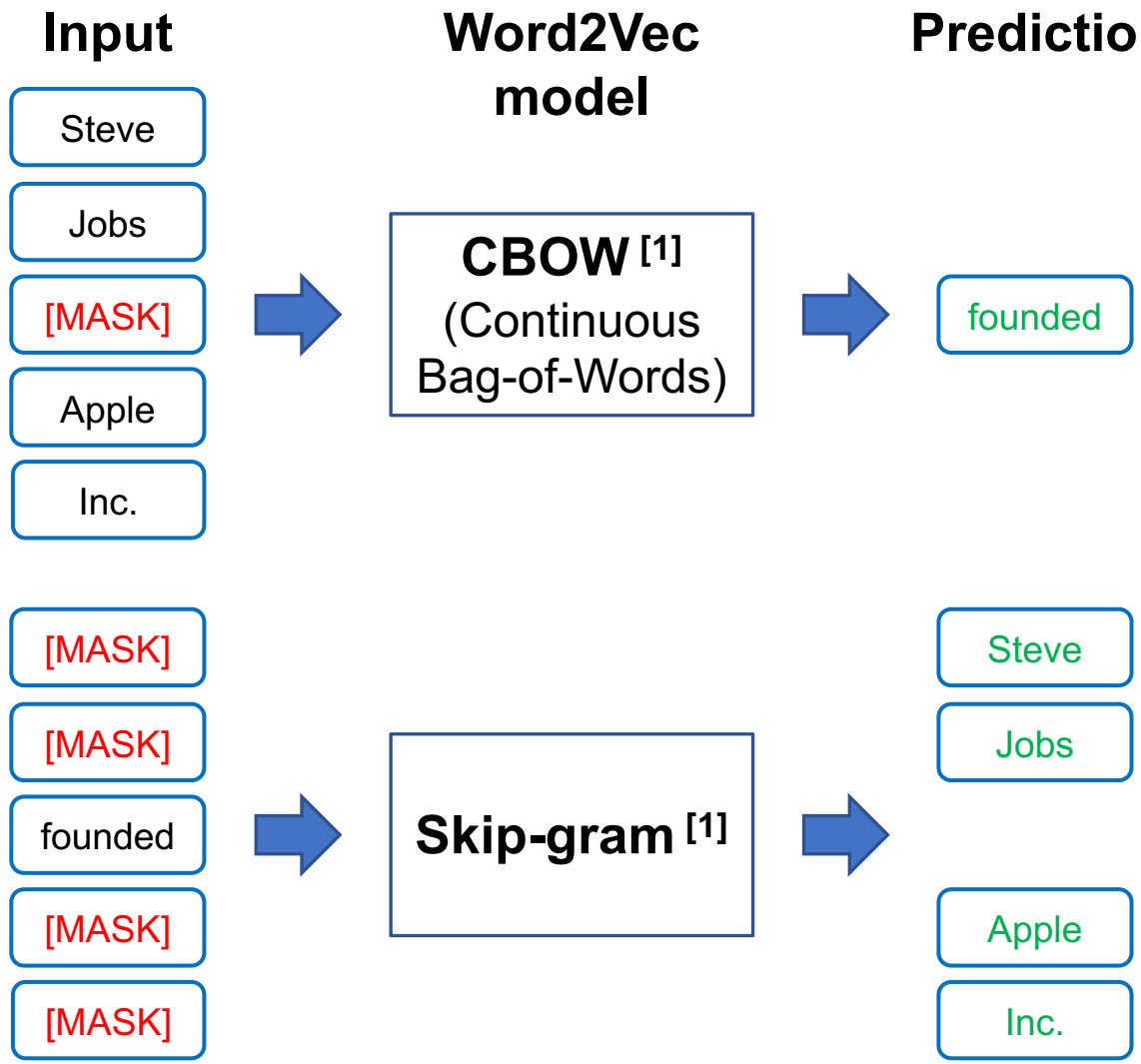


Word Embeddings (Word2Vec)

[1] Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013.

Large Corpora (E.g., Wikipedia)





After Training Word Embeddings

- The outputs are dense vectors with a fixed dimension size:

Dimension size (e.g., 300)

Vocabulary size (e.g., 30,000)

apple -0.110960 0.016115 -0.004809 0.033589 0.121455 …
banana -0.027713 -0.015676 0.003314 0.077602 0.159718 …
…
…
…



NLP 三大秘辛

1. 每個字，都是一段向量
2. 每個 NLP 模型都有一個字典 
3. 自然語言處理模型需要記憶力 



每個 NLP 模型都有一個字典 

Vocabulary

- apple
- eat
- I
- like
- watch
- to
- ⋮

→ LM → I like to eat apple watch

字典裡面的內容是語言模型能理解的最小單元，
如此一來模型才能夠進行目標任務 (e.g., 生成)



→ → ... (你平常操作 ChatGPT 的結果)

字典實作

字典的變數
(key: 字, value: index)

通常我們會預先準備 <pad> 跟 <unk>
前者代表 padding，後者代表 unknown words

```python
vocab = {'<pad>':0, '<unk>':1}

# 使字詞不重複
words = sorted(set(words))
for idx, word in enumerate(words):
    # 一開始已經放兩個進去 dictionary 了
    idx = idx + 2
    # 將 word to id 放到 dictionary
    vocab[word] = idx
```

Python

字典在NLP中的實作方式就真的採用Python dictionary



[注意事項] 字典實作

字典是經由訓練資料集的字所建立的

```
word_to_idx = {"<pad>": 0, "<unk>": 1, "好吃": 2, "棒": 3, "给力": 4}
embeds = torch.nn.Embedding(5, 5)  # 5 words in vocab, 5 dimensional embeddings
```

```
def get_word_id(word, vocab, unk_idx: int = 1):
    return vocab.get(word, unk_idx)
```

如果某字不存在於字典中，則通常給予 <unk> 的 id

```
lookup_tensor = torch.tensor(
    [
        get_word_id("好吃", word_to_idx),
        get_word_id("不棒", word_to_idx),
        get_word_id("<unk>", word_to_idx),
    ],
)
word_embed = embeds(lookup_tensor)
print(word_embed)
```

```
tensor([[-0.5574,  0.1768,  0.9588,  1.2837, -1.7256],
        [-3.4725, -1.0435, -0.0538, -0.0625, -0.4173],
        [-3.4725, -1.0435, -0.0538, -0.0625, -0.4173]],
       grad_fn=<EmbeddingBackward0>)
```



NLP 三大秘辛

1. 每個字，都是一段向量
2. 每個 NLP 模型都有一個字典 
3. 自然語言處理模型需要記憶力 🧠

語言的模式

⭐ 上下文非常重要

句子1：

小明昨天遇到小美，他對她說：「下次一起去看電影吧！」，請問小明下次想去跟誰看電影？

句子2：

「庭院深深深幾許」，請問有幾個深？

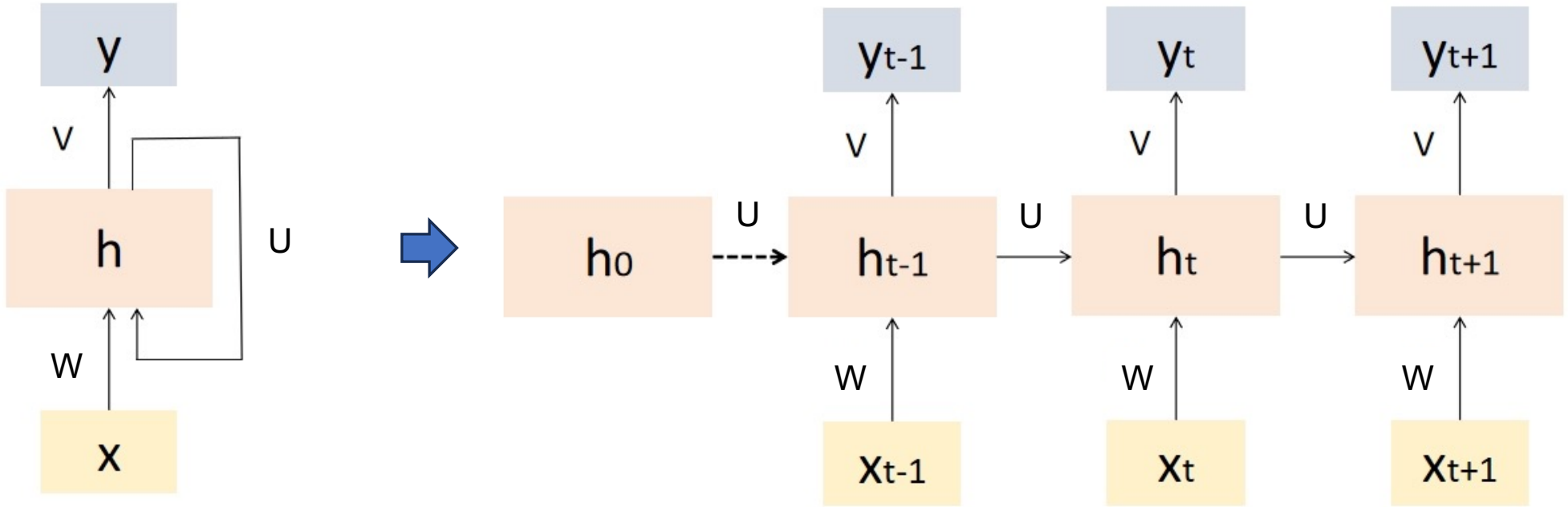


序列模型

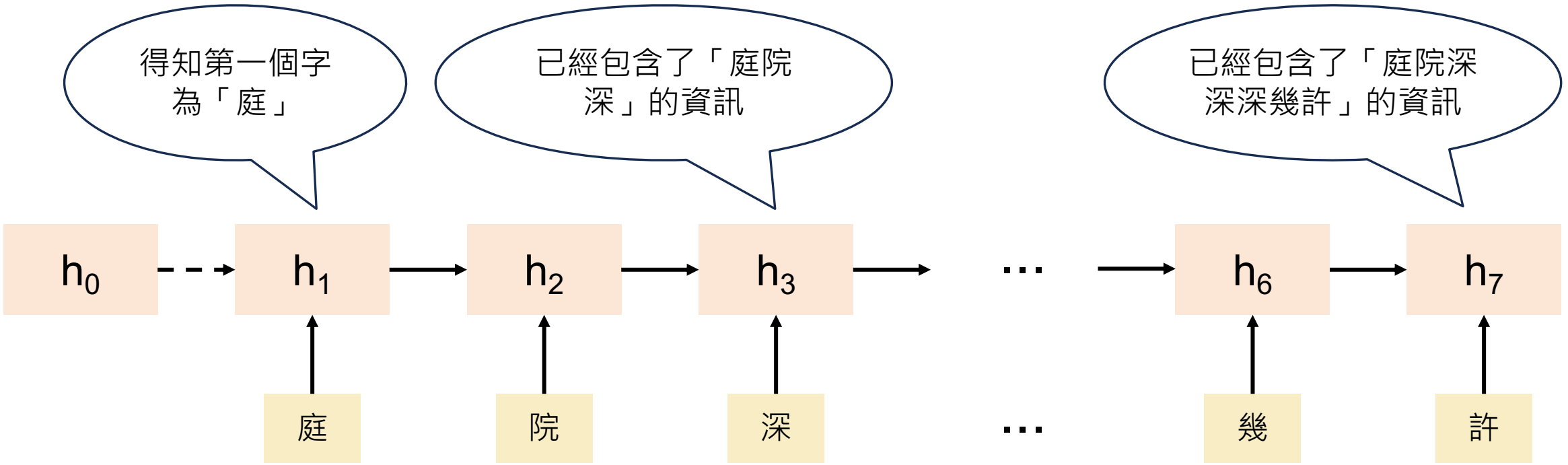
RECURRENT NEURAL NETWORKS (RNN)

Recurrent Neural Networks (RNN)

- Recurrent Neural Networks (RNNs) are a type of artificial neural network designed to handle sequential data by capturing temporal dependencies.
  - RNN 的“遞迴”是指它在每一個時間步中重複使用相同的參數（同一組權重），並把先前的隱藏狀態傳到下一個時間步



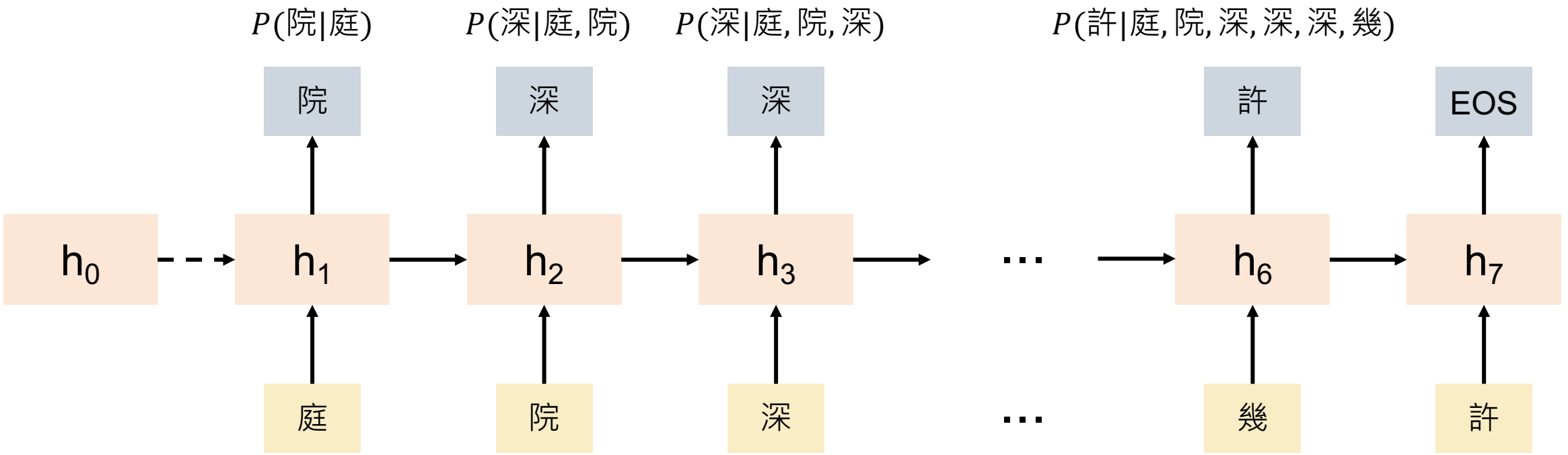
RNN 的記憶力意義


得知第一個字為「庭」
已經包含了「庭院深」的資訊
已經包含了「庭院深深深幾許」的資訊
h_0
h_1
h_2
h_3
…
h_6
h_7
庭
院
深
…
幾
許

RNN 以文字接龍進行訓練

EOS: End of Sentence

$P(\text{EOS}|\text{庭, 院, 深, 深, 深, 幾, 許})$



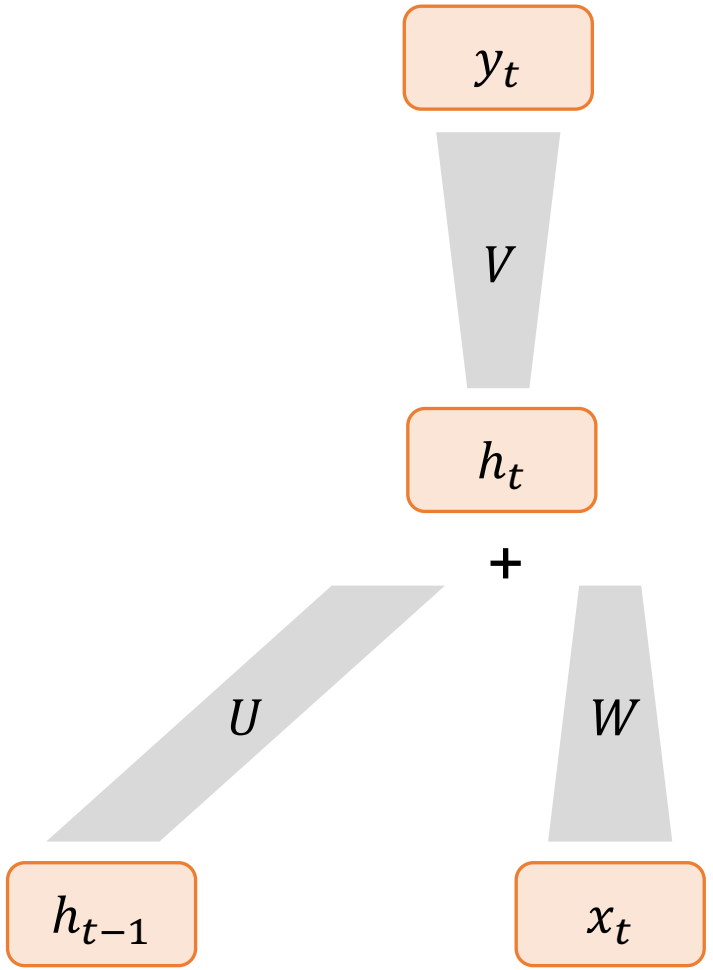
目標函數：log($P(\text{院}|\text{庭})\times P(\text{深}|\text{庭, 院})\times P(\text{深}|\text{庭, 院, 深})\times \ldots \times P(\text{EOS}|\text{庭, 院, 深, 深, 深, 幾, 許})$)

RNN (本頁省略 bias)

- The equation on step t is:

$$h_t = \tanh(Wx_t + Uh_{t-1})$$
$$y_t = Vh_t$$

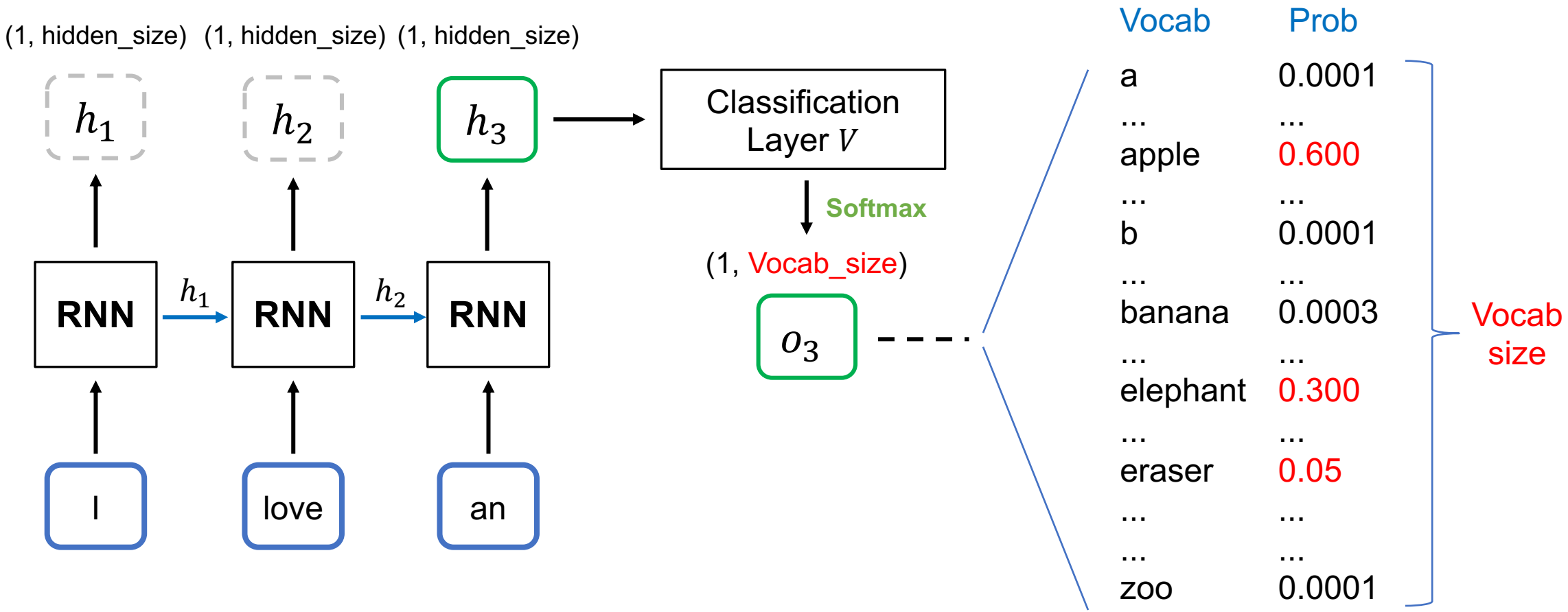
, where tanh is an activation function.





Text Generation with RNN

*本頁的RNN不包含輸出層V



Word embeddings

Why is softmax called softmax?

soft: 機率 (帶有資訊量)

Max:



1



0

Softmax:



0.6

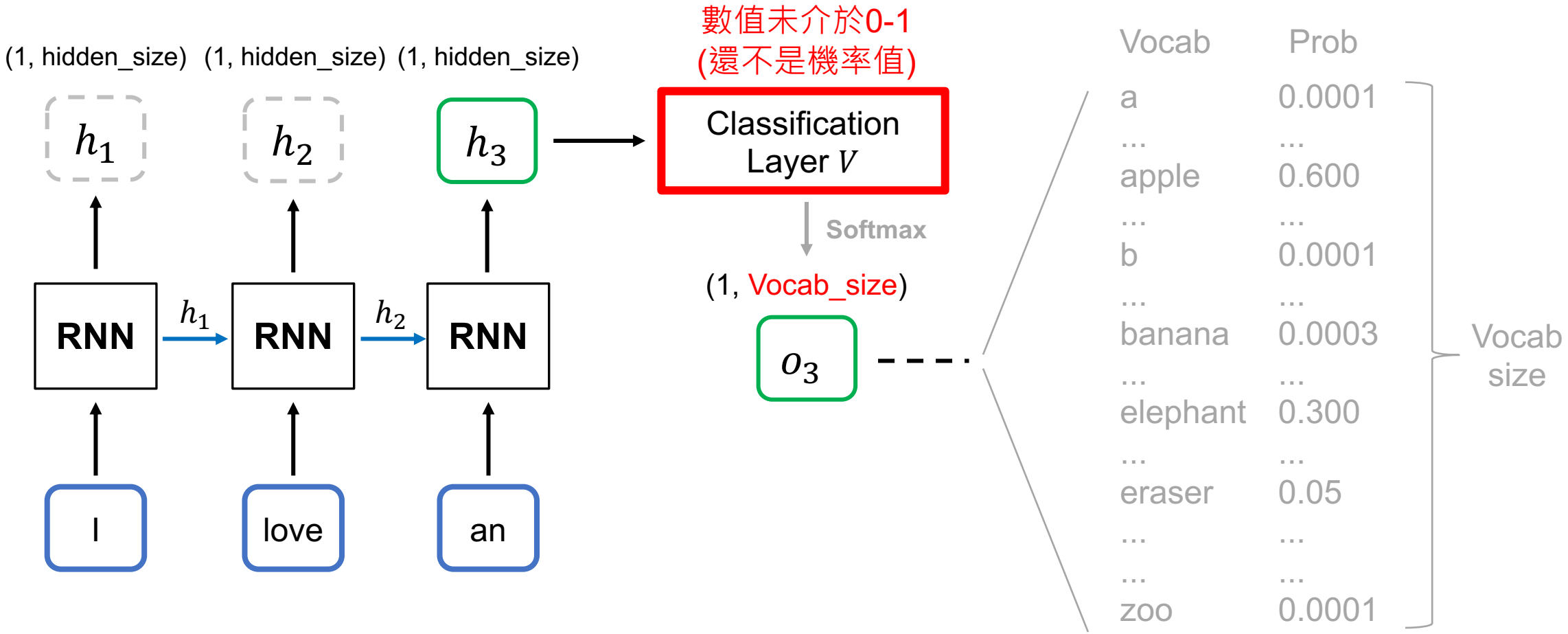


0.4



Text Generation with RNN (V輸出時)

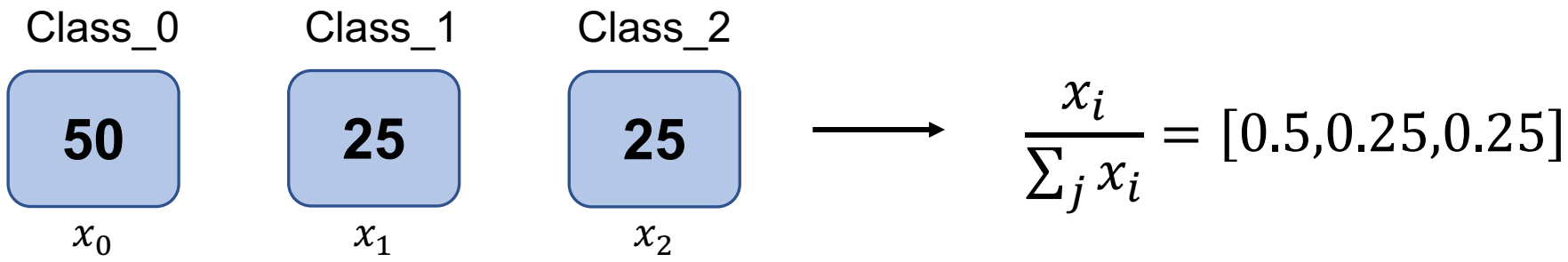
*本頁的RNN不包含輸出層V



Word embeddings

[Recap] 如何把模型輸出變成機率值 (0~1)？

機率總和為1，每個機率介於 0 ~ 1 之間



- **Softmax** function，採用 exponential: $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$
  1. 恆正（符號不變性）
  2. 大的數值更大，小的數值更小，有助於梯度下降

注意！Softmax 有考慮到類別總和

[Recap] 為什麼恆正重要？

機率總和為1，每個機率介於 0 ~ 1 之間

Class_0	Class_1	Class_2
50	-25	25
x_0	x_1	x_2

$$\frac{x}{\sum_j x_j} = [1.0, -0.5, 0.5]$$

Class_0	Class_1	Class_2
50	-25	25
x_0	x_1	x_2

$$\text{Softmax}(x_i) = \frac{e_i^x}{\sum_j e^{x_j}}$$

$$= [1.0, 2.6786e-33, 1.3888e-11]$$



[Recap] Softmax 帶來大者恆大的效應

```
import torch
from torch.nn.functional import softmax

logits = torch.tensor([50, 25, 25], dtype=torch.float32)
print(softmax(logits, dim=0))
```

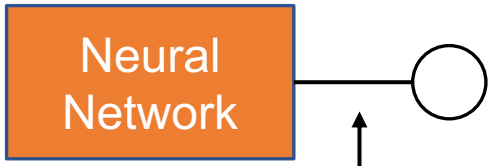
```
tensor([1.0000e+00, 1.3888e-11, 1.3888e-11])
```

```
# logits = torch.tensor([5, 2, 2], dtype=torch.float32)
tensor([0.9094, 0.0453, 0.0453])
```



[Recap] 如何對一個模型輸出值取機率值(0~1)？

- **Sigmoid** function:



假設閾值為0.5

> 0.5 是貓，
否則不是貓

$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$$

x代表logit
x很大代表e^{-x}很小
-> $\text{Sigmoid}(x)$很大

注意！sigmoid 只需要1個數值就能計算

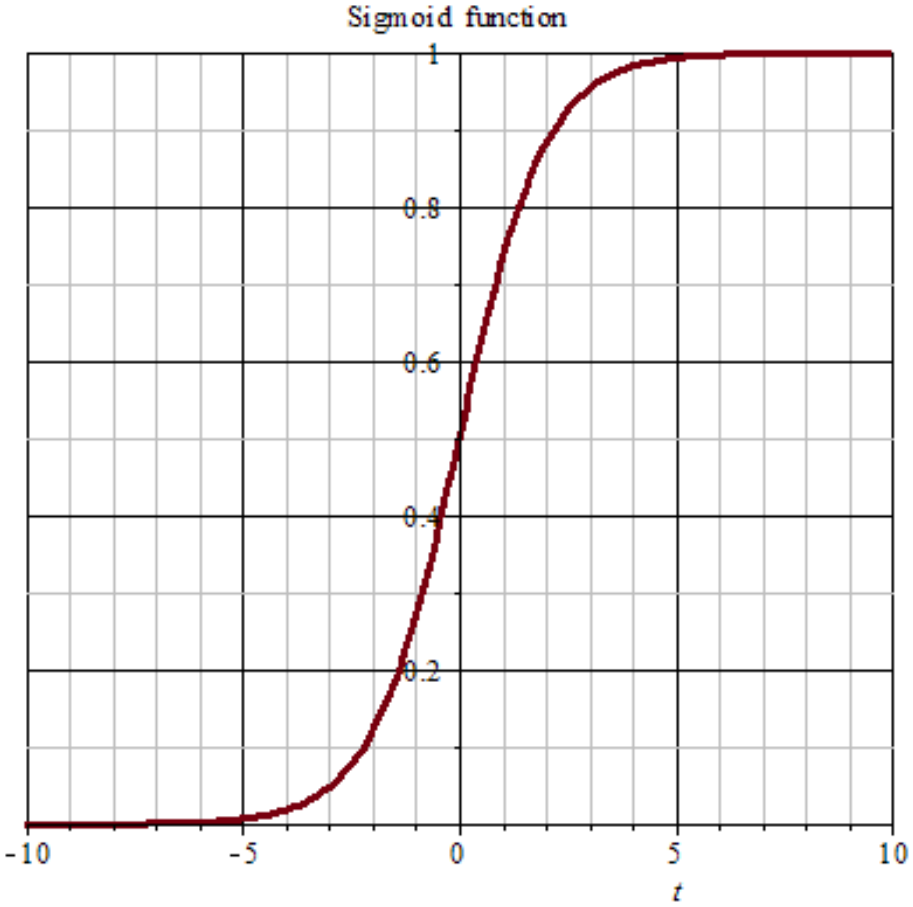
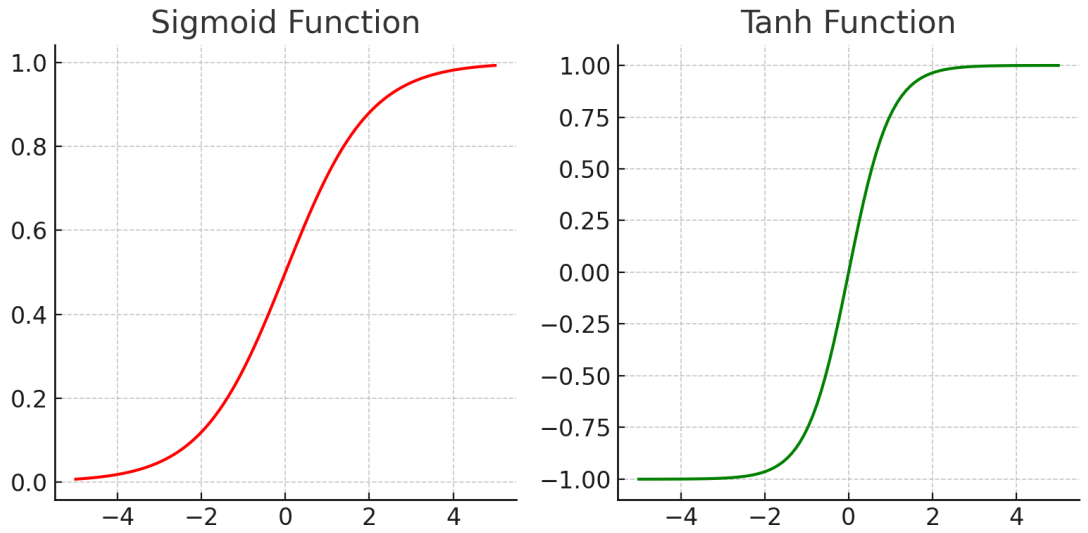


Figure source: https://zh.wikipedia.org/zh-tw/S型函數

RNN 使用 tanh (hyperbolic tangent)

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



RNN 作為序列模型，需要在多個時間步傳遞 hidden state 與梯度。相較於 Sigmoid，tanh 具有以下優點：

1. Zero-centered： 輸出介於 −1 到 1，可表達正負方向的特徵。
2. 較大的梯度： tanh 在接近 0 的區域具有較大的導數，有利於梯度傳遞。



Softmax: https://datascience.stackexchange.com/questions/57005/why-there-is-no-exact-picture-of-softmax-activation-function

Activation functions (啟動函數)

	值的範圍	使用情境
softmax	[0, 1] (sum up to 1)	需要讓多個值產生相加為一的機率值的時候，常用於建立分類任務模型時計算 cross-entropy loss
sigmoid	[0, 1]	需要讓單一值產生機率值的時候，常用於建立二元分類任務模型時計算 binary cross-entropy loss
tanh	[-1, 1]	常用於LSTM等RNN模型中，主要為引入非線性轉換。 tanh的值是對稱的，以零為中心，tanh 在接近 0 的區域具有較大的導數，和sigmoid相比較可能不會梯度消失
ReLU	[0, 正值]	常用於CNN或MLP等模型中，主要為引入非線性轉換。 計算簡單，在許多任務收斂快

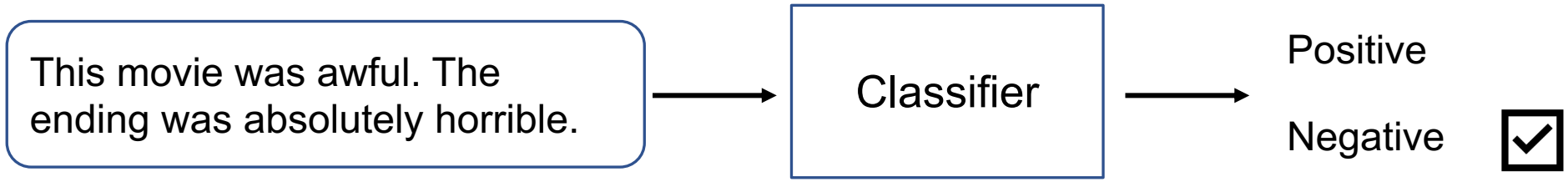


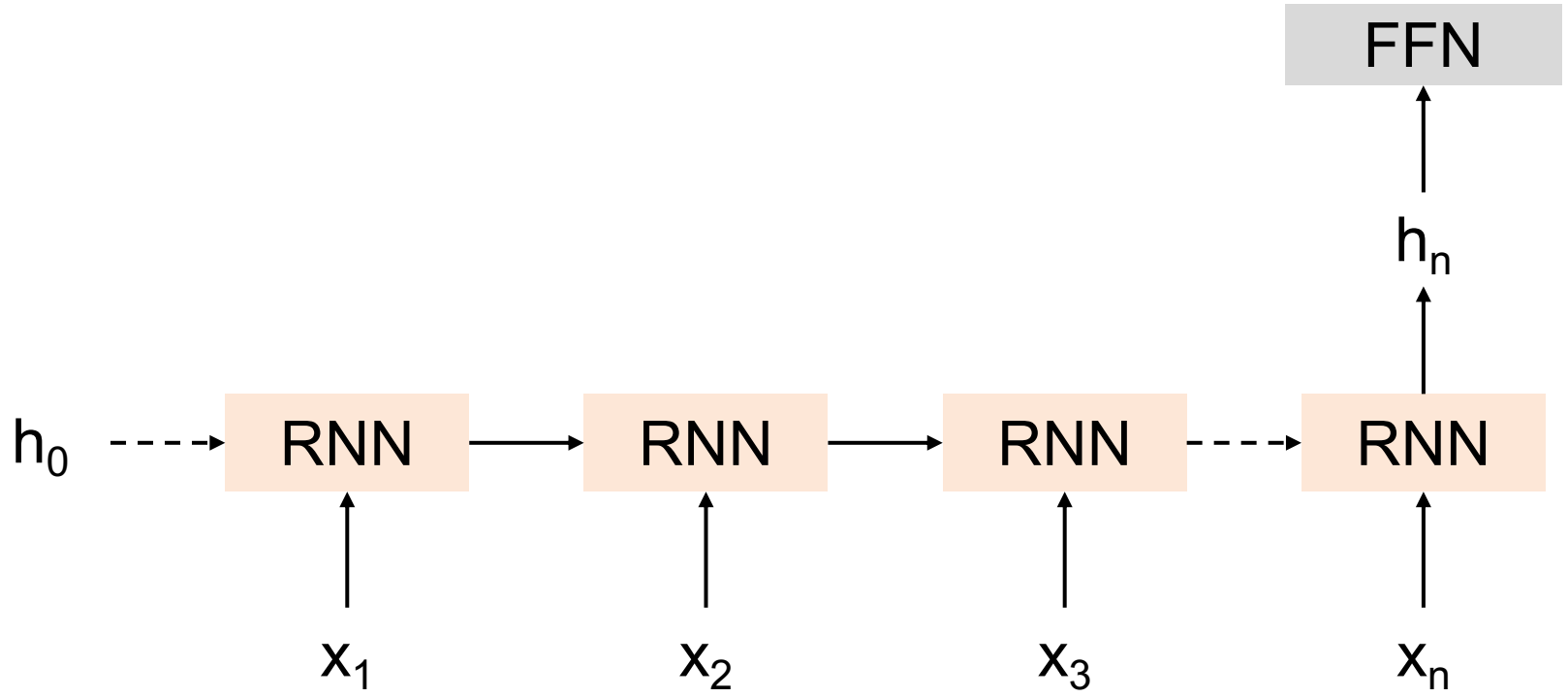
為什麼我們通常不用 MLP 來做 NLP？

1. **無法處理序列順序**
   - MLP 不會記住序列的前後順序 (上下文關係)，但語言的順序非常重要。
2. **輸入長度必須固定**
   - MLP 需要固定長度的輸入，但句子的長度通常不同。



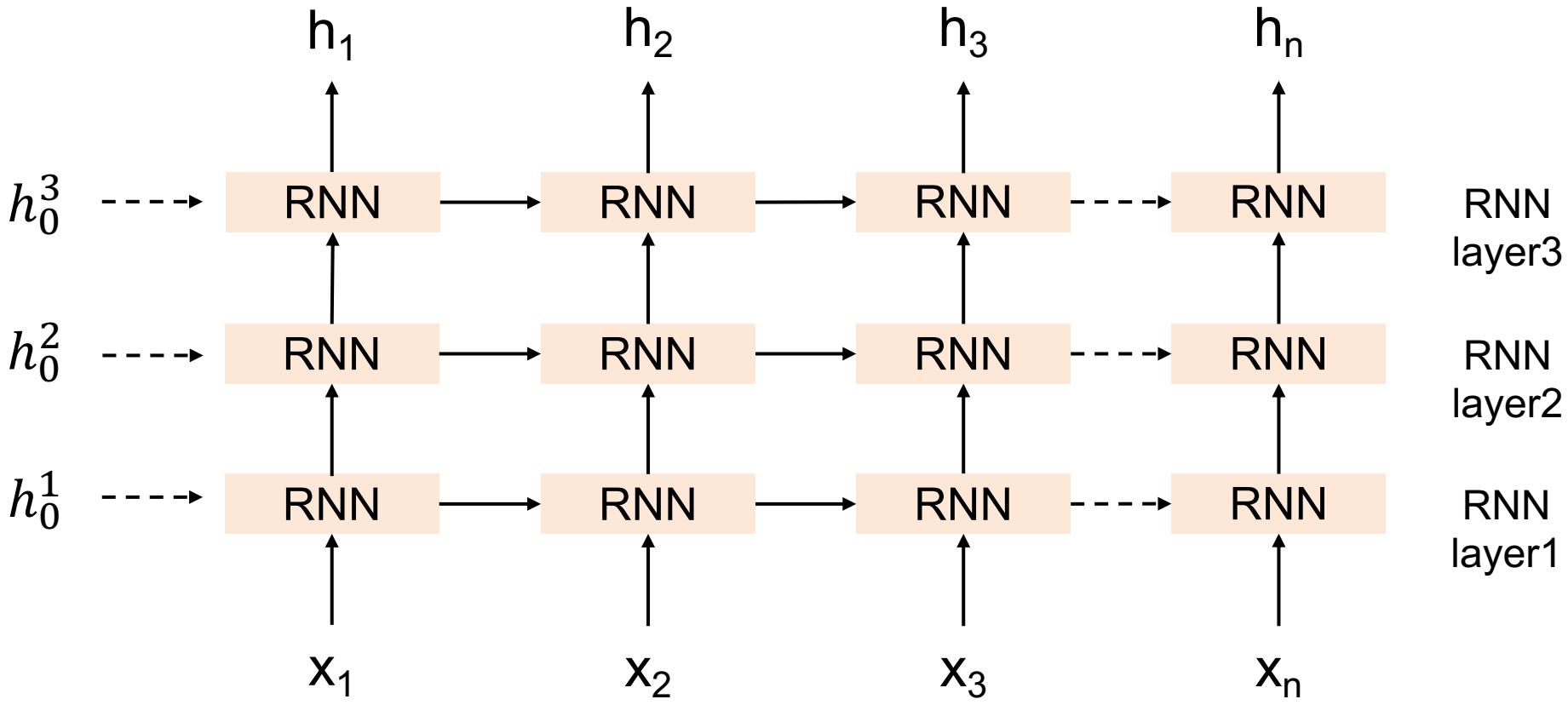
RNNs for Sequence Classification





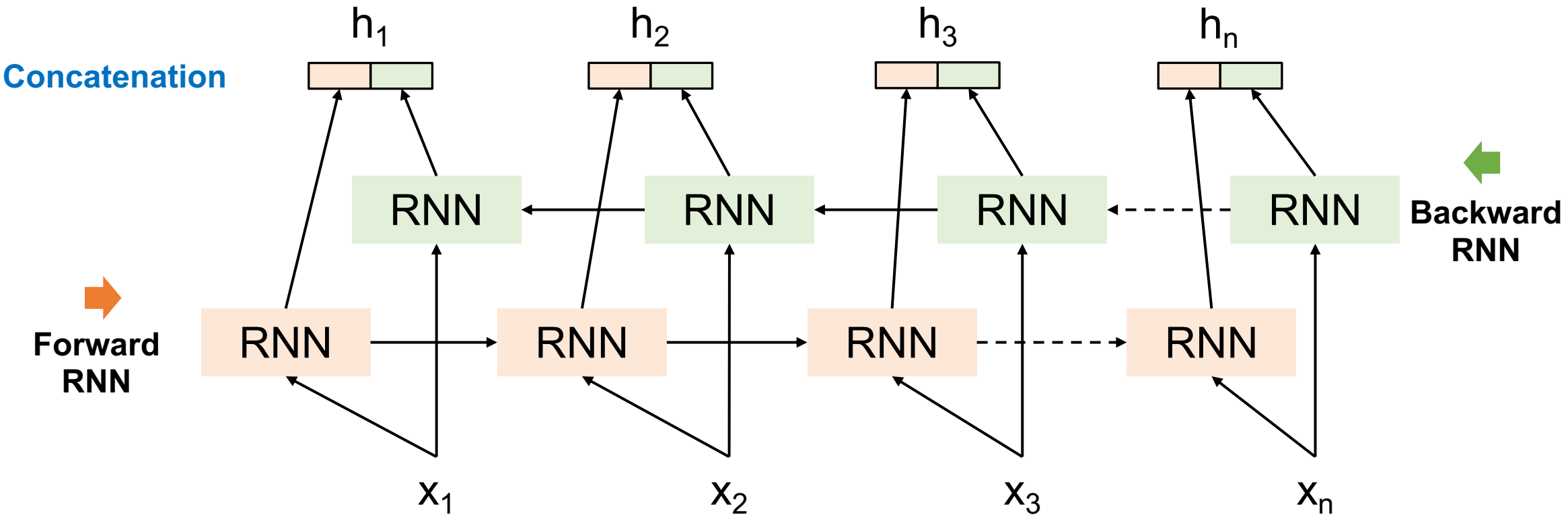
Stacked RNNs

- Stacked RNNs consist of multiple networks where the output of one layer serves as the input to a subsequent layer



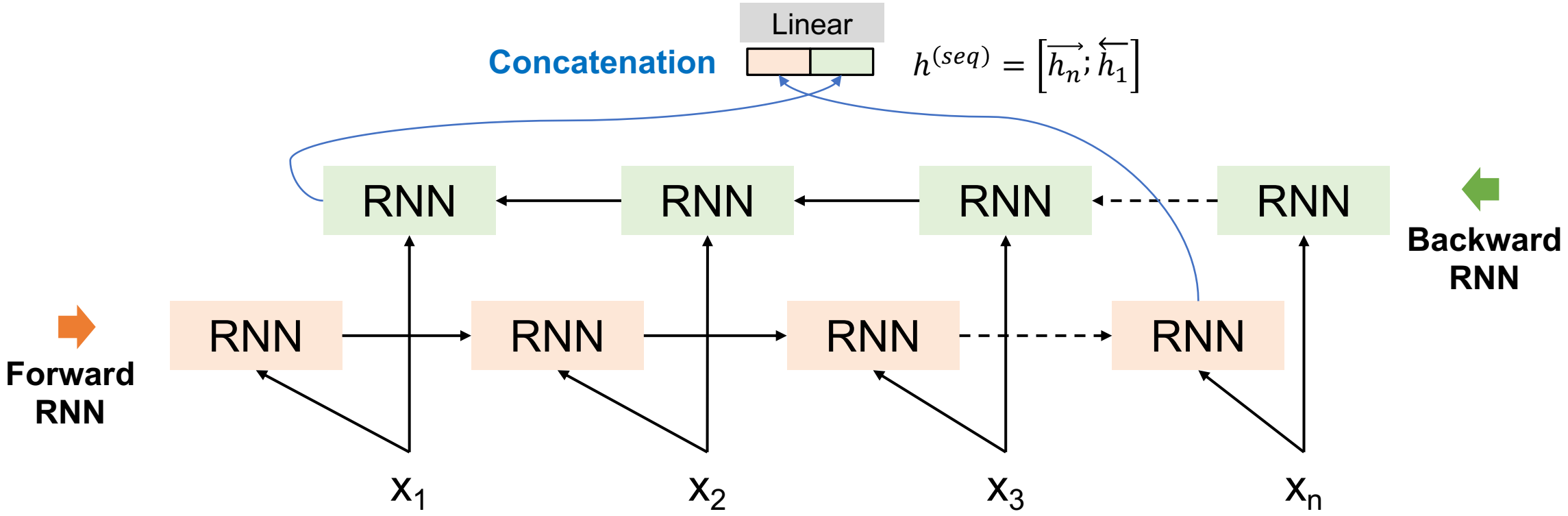
Bidirectional RNNs

$$h_t = \left[\overrightarrow{h_t}; \overleftarrow{h_t}\right]$$



Bidirectional RNNs for Sequence Classification

- The final hidden units from the forward and backward passes are combined to represent the entire sequence.
- This combined representation serves as input to the subsequent classifier.

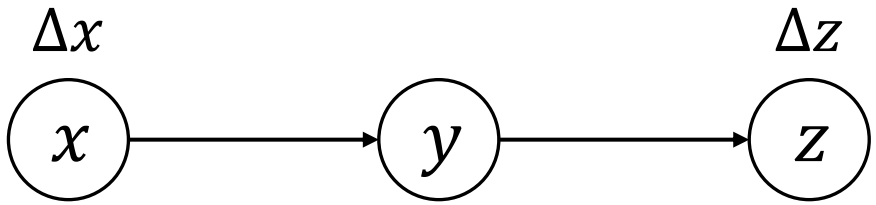


Shortage of standard RNNs

- Gradient vanishing (梯度消失) problem
- Long-term dependencies cannot be handled.
  - RNNs can only remember local relationships.
  - This is result from back-propagation through-time (BPTT) during training.



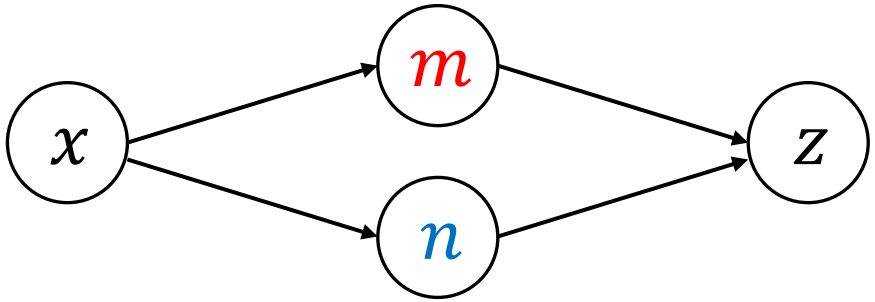
(Calculus) Chain Rule - 1



$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

- $\frac{dz}{dx}$ 代表 x 對 z 的影響，但 x 的變化會影響到 y ，接著 y 影響到 z

(Calculus) Chain Rule - 2



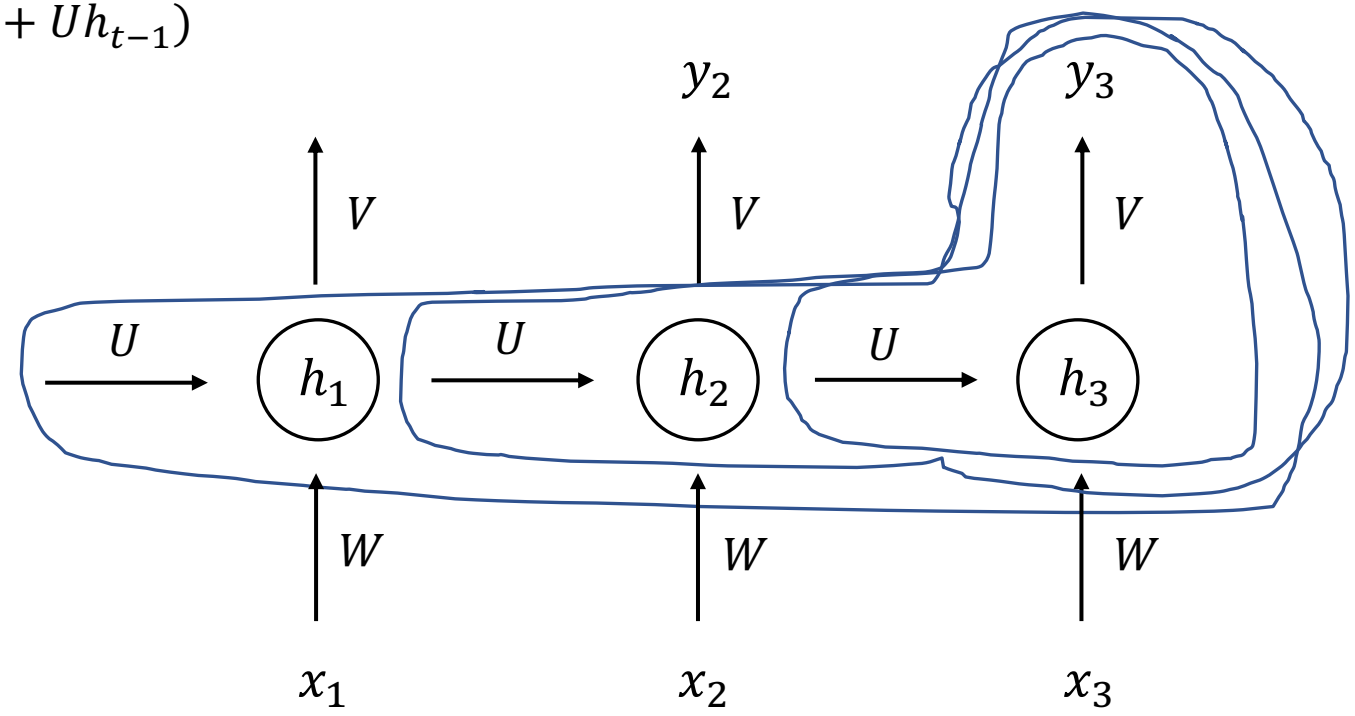
$$\frac{dz}{dx} = \frac{dz}{dm}\frac{dm}{dx} + \frac{dz}{dn}\frac{dn}{dx}$$

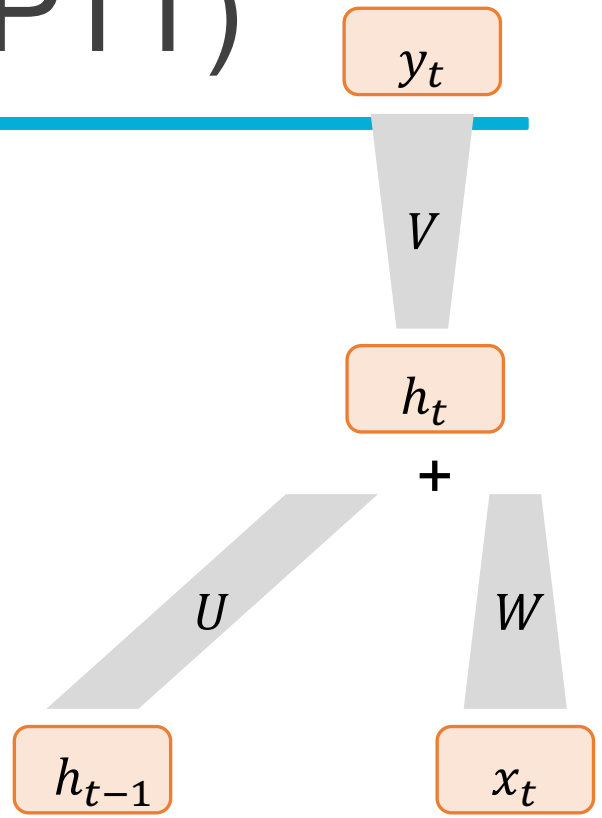
- $\frac{dz}{dx}$ 代表 x 對 z 的影響
  - 但 z 的變化由 m 和 n 共同影響



Back-propagation through-time (BPTT)

$h_t = \tanh(Wx_t + Uh_{t-1})$
$y_t = Vh_t$





$$\frac{\partial \mathcal{L}}{\partial U} = \frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial U} + \frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial U} + \underbrace{\frac{\partial \mathcal{L}}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial U}}_{\text{(small)}}$$

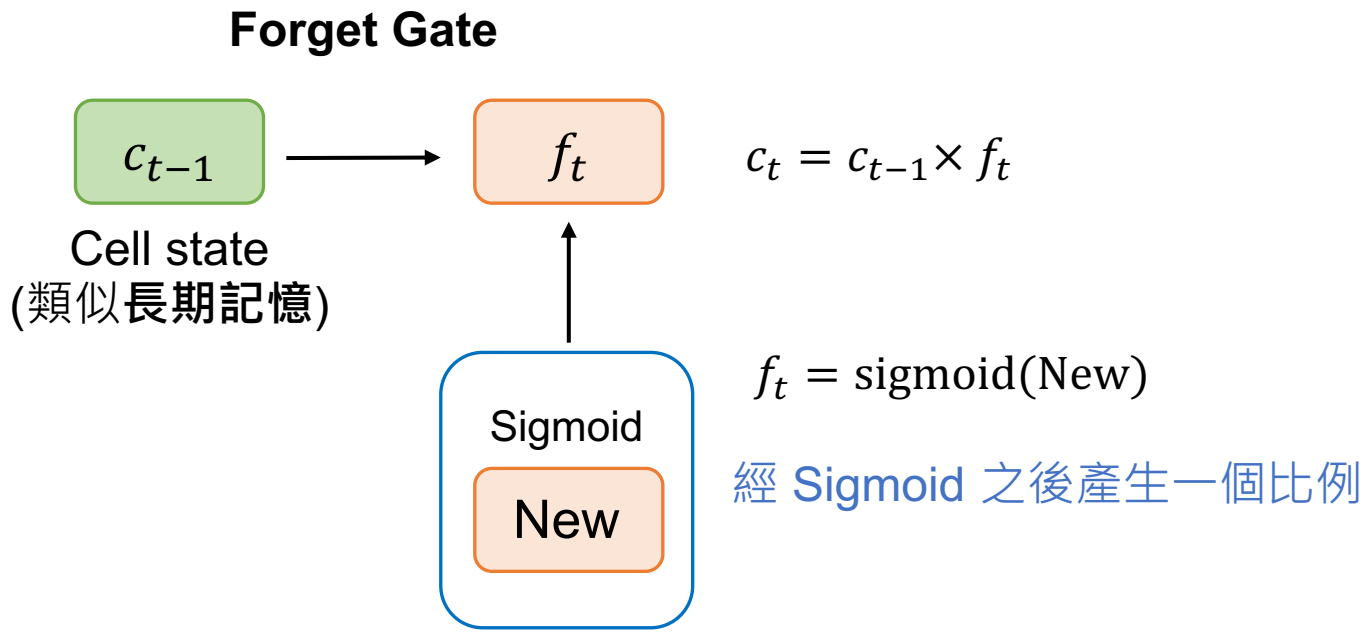


Long Short-Term Memory

LSTM

Gate (以 Forget Gate 為例)

LSTM 新增了 Cell State

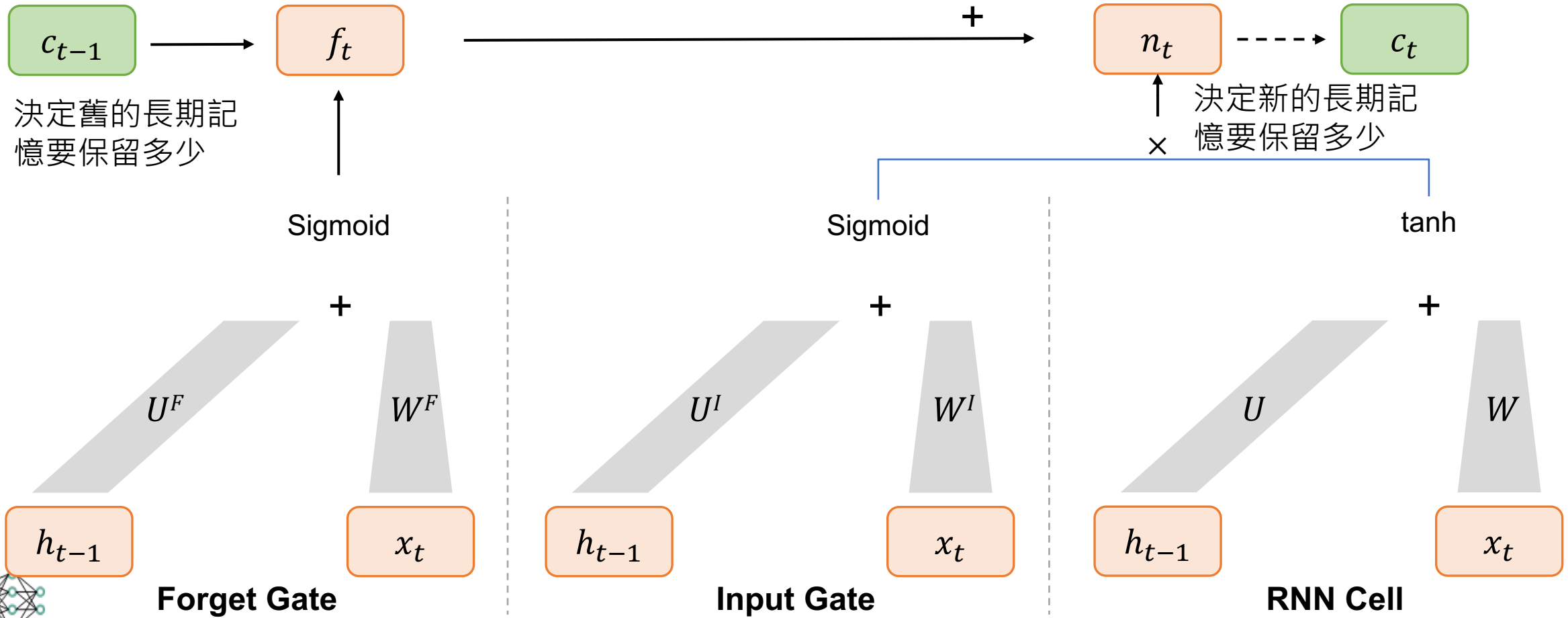


Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." Neural computation 9.8 (1997): 1735-1780.

LSTM (Long Short-Term Memory)

➢ LSTM uses 4 times of weights as a standard RNN.

$$c_t = c_{t-1} \times f_t + n_t$$

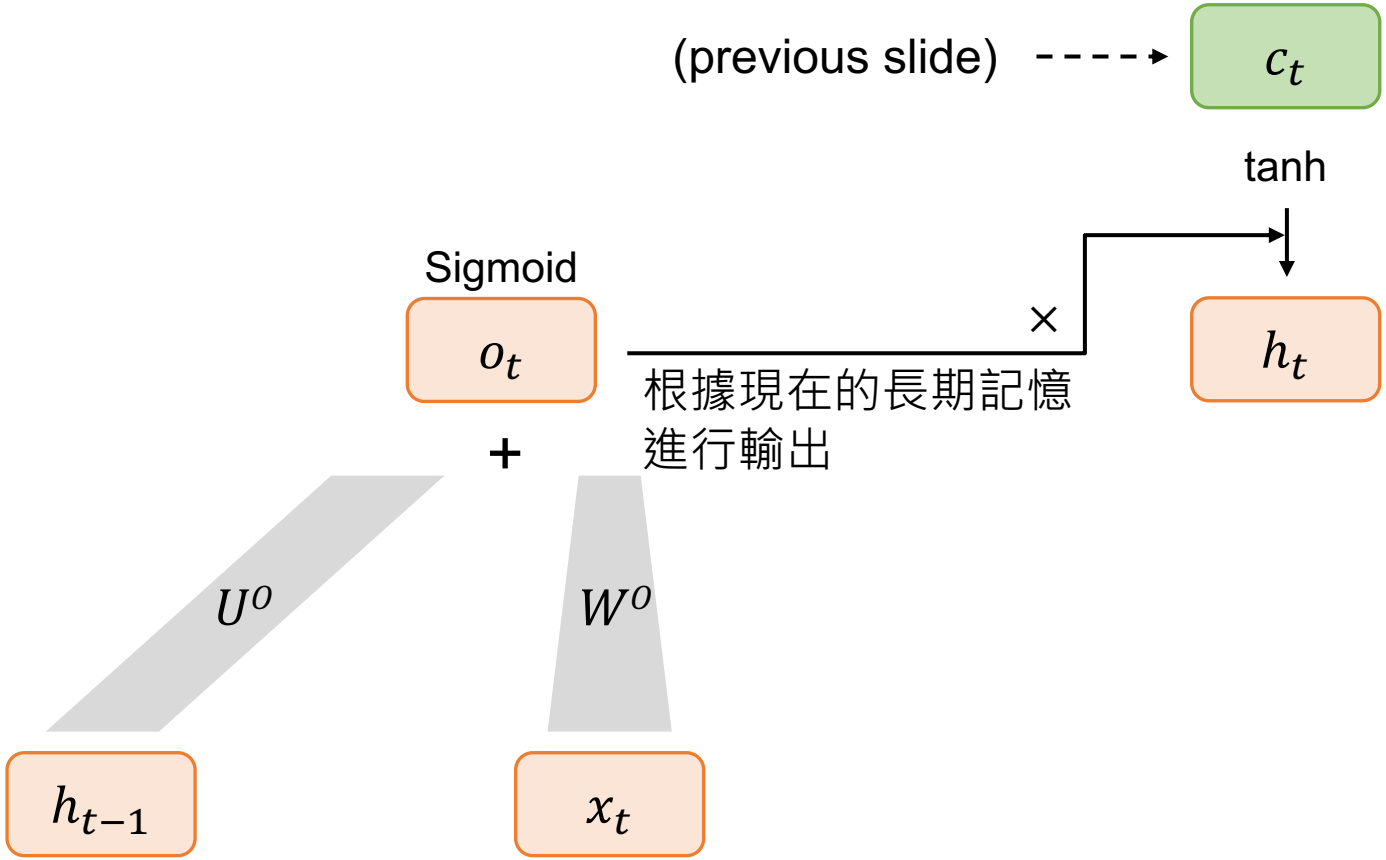


Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." Neural computation 9.8 (1997): 1735-1780.

LSTM (Long Short-Term Memory)

- LSTM uses 4 times of weights as a standard RNN.

$$c_t = c_{t-1} \times f_t + n_t$$



Output Gate

延伸學習

https://colah.github.io/posts/2015-08-Understanding-LSTMs/



RNN vs. LSTM

	Memory Mechanism	Handles Long Dependencies?	Gradient Vanishing?
RNN	Hidden state h_t	Worse	Yes
LSTM	Memory state c_t + Hidden state h_t	Better	Much less than a standard RNN



期中考提醒 (11/04 13:10 - 16:00)

- 教室分兩間，座位表將於考試當日上午公布
- 類型為：
  - 觀念題 (問答為主、選擇為輔)
  - 紙上上機：與模型訓練或深度學習函式庫有關的程式碼細節
- 考試範圍：Week1 - Week8
  - 也就是 GitHub repo 上面的所有內容、作業與小考題
- 期中考佔學期成績比例 30%



Thank you!

Instructor: 林英嘉

✉ yjlin@cgu.edu.tw

TA: 鄭茹云

✉ m1461010@cgu.edu.tw